Tratamento de Exceções

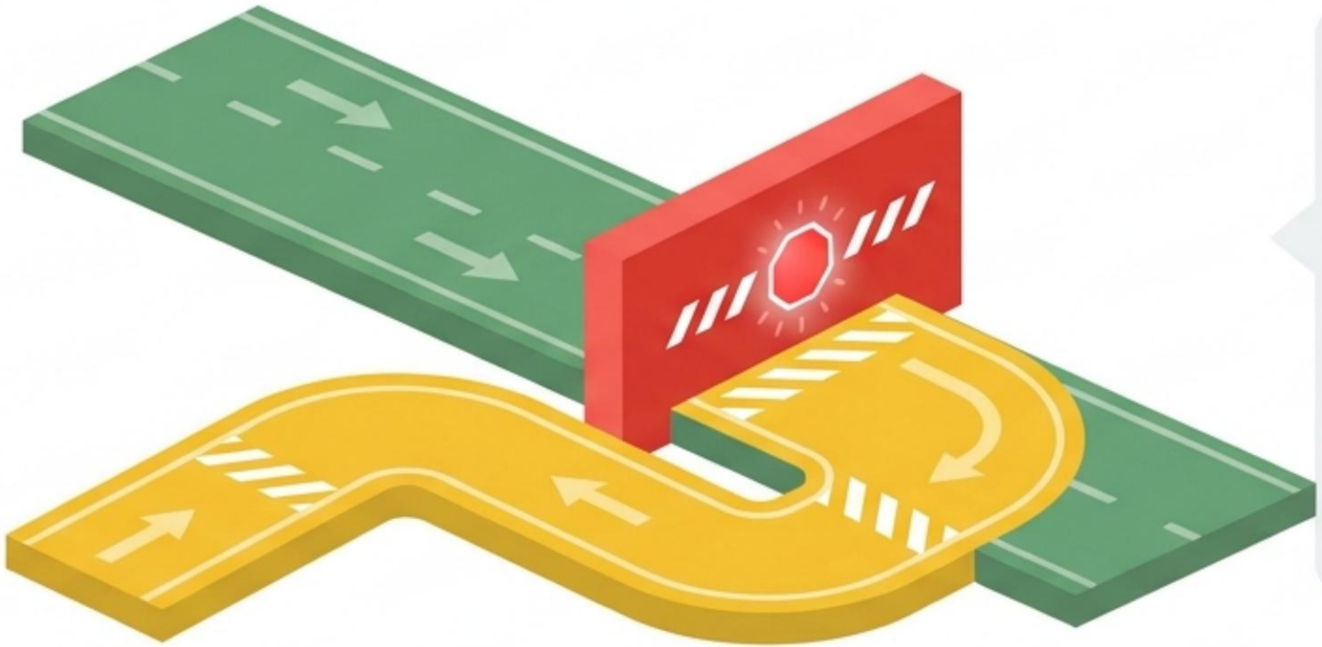
Aula 10: Try-catch, multicatch, finally, throw e throws

Prof. Ronaldo Borges



NotebookLM

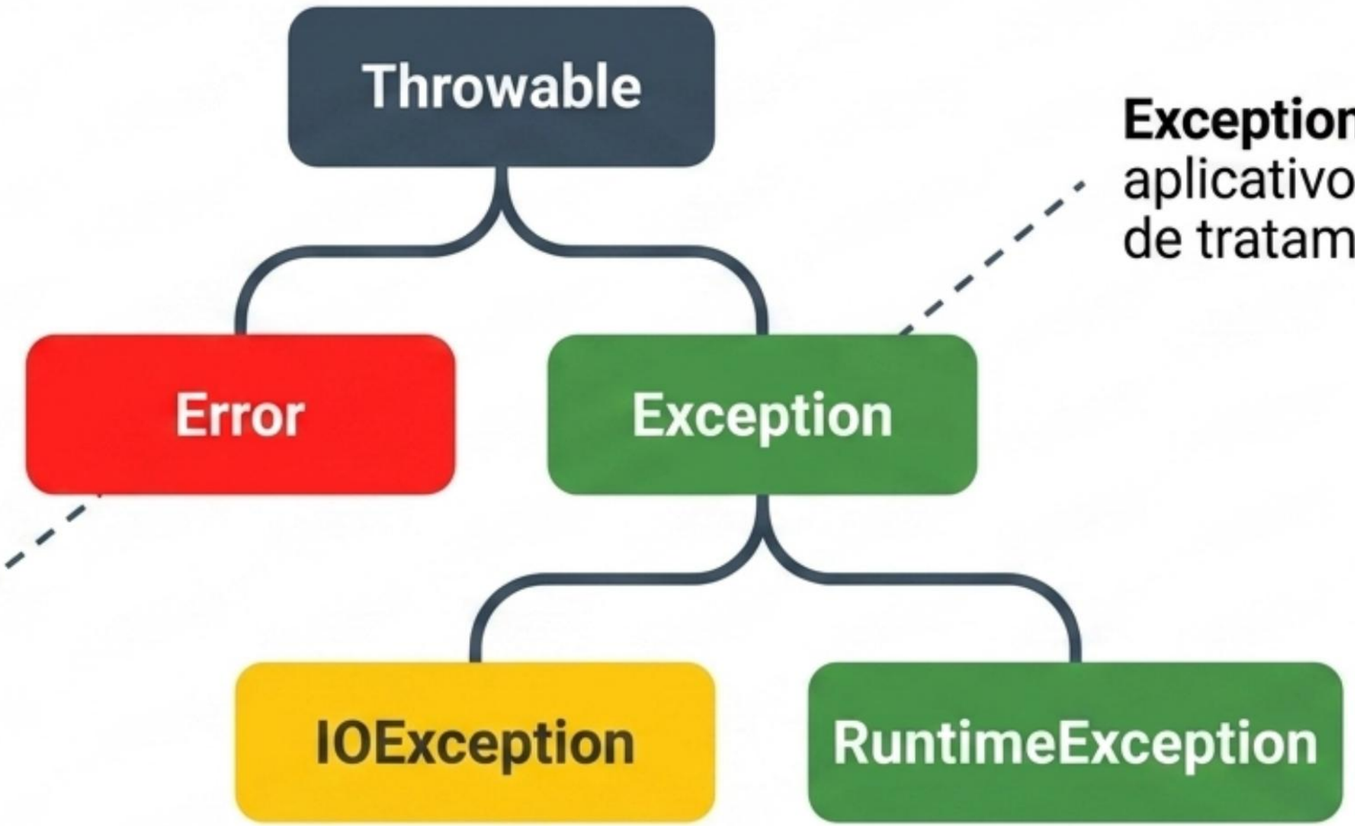
O Paradigma do Tratamento de Exceções



Uma exceção é um evento que interrompe o fluxo normal de instruções do programa. O tratamento de **exceções não evita o erro,** mas permite que o sistema seja **tolerante a falhas, criando rotas alternativas em vez de um travamento (crash)** (crash) imediato.

NotebookLM

A Árvore Genealógica das Exceções



Exception: Erros do aplicativo. Nosso foco de tratamento!

Error: Falhas críticas na JVM (ex: falta de memória). Não devem ser capturados pelo aplicativo.

NotebookLM

Matriz de Diagnóstico: Verificadas vs. Não Verificadas

Exceções Verificadas (Checked) ⚠️	Exceções Não Verificadas (Unchecked) ⛔
Condições externas ao programa (ex: arquivo ausente, falha de rede).	Erros de lógica de programação (bugs estruturais).
O compilador Java exige que sejam tratadas.	O compilador não exige o tratamento explícito.
Subclasses diretas de Exception.	Subclasses de RuntimeException.
IOException, FileNotFoundException.	NullPointerException, ArithmeticException.

NotebookLM

A Rede de Segurança: A Sintaxe Try-Catch



A Zona de Risco: Onde o código monitorado é executar. Se ocorer um erro, a execução é interrompida imediatamente.

```
try {
    // Código que pode falhar
} catch (TipoDaExcecao e) {
    // O que fazer se falhar
}
```



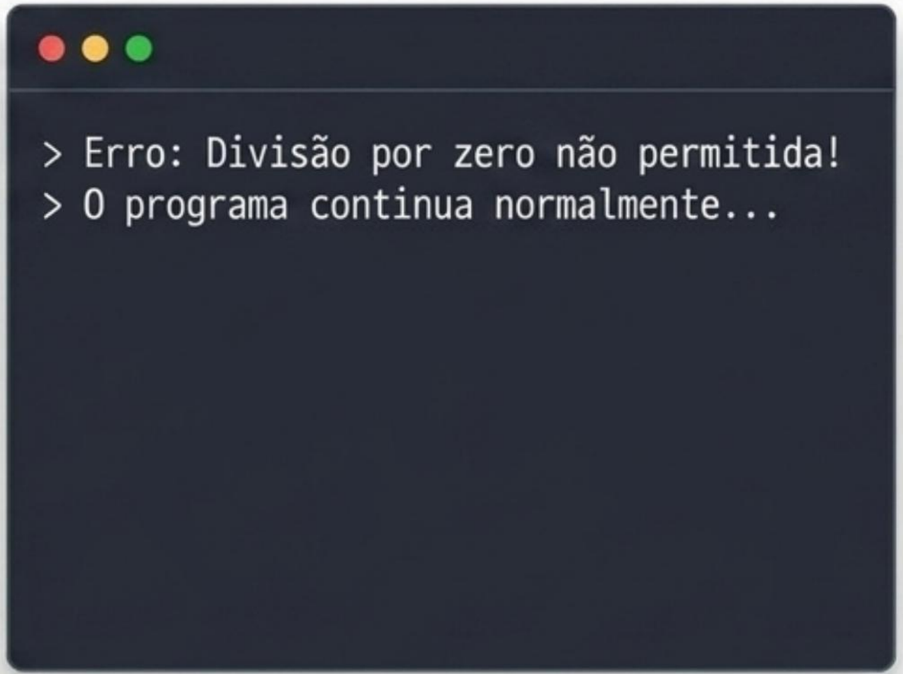
A Rede de Tratamento: O desvio de emergência. Só é executado se a exceção especificada for lançada no bloco try.

ronaldo.pb@ifpi.edu.br | (SCHILDT, 2015)

NotebookLM

Exemplo Prático: Try-Catch em Ação

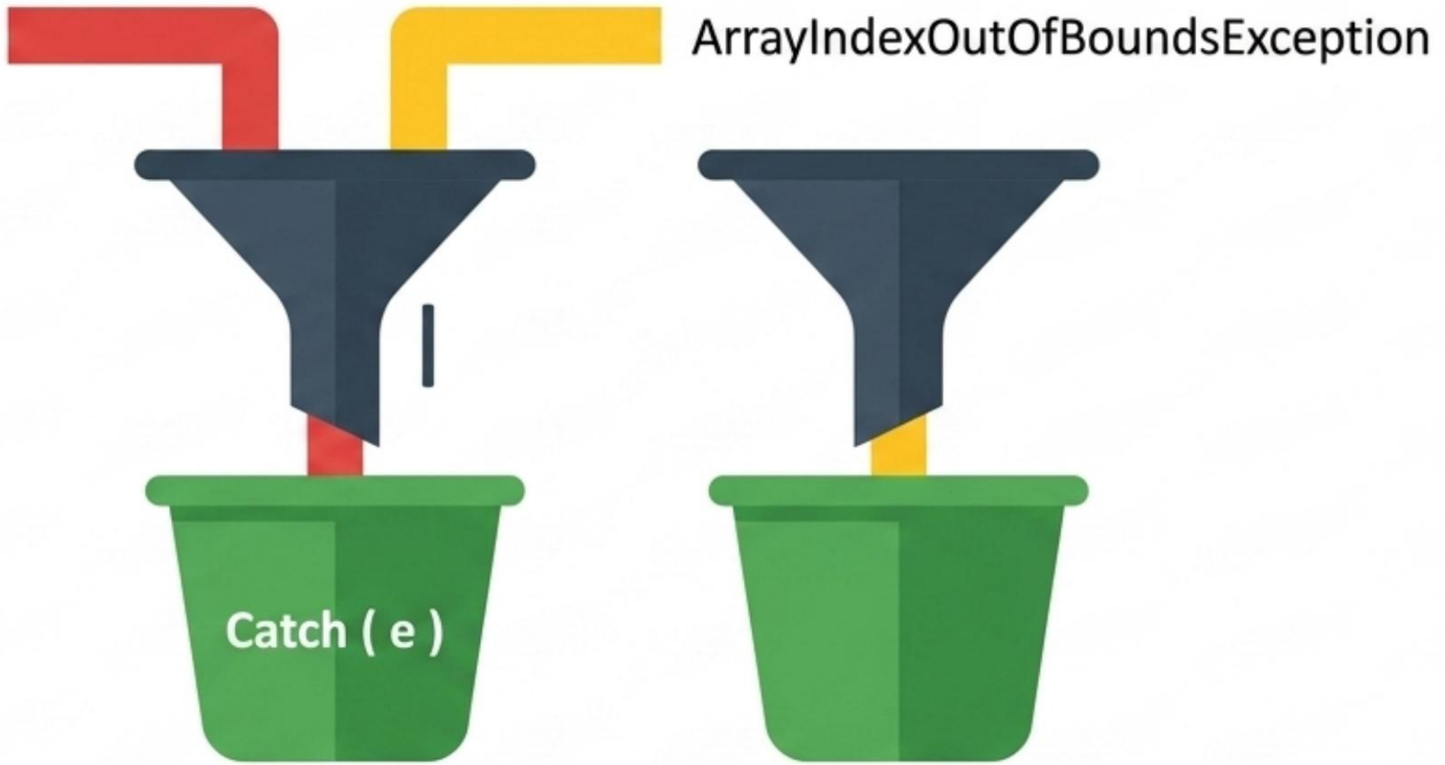
```
int a = 10;
int b = 0;
try {
    int resultado = a / b;
    System.out.println(resultado); // Não executará
} catch (ArithmeticException e) {
    System.out.println("Erro: Divisão por zero não
permitida!");
}
System.out.println("O programa continua
normalmente...");
```



NotebookLM

Elegância e Otimização: O Recurso Multicatch

Introduzido no Java 7, o multicatch permite que uma única cláusula **catch** capture duas ou mais exceções diferentes.



A Regra de Ouro : Utilize quando diferentes exceções exigirem exatamente a mesma ação corretiva. O parâmetro da exceção torna-se implicitamente final.

NotebookLM

Exemplo Prático: Multicatch

```
int[] numeros = {1, 2, 3};
int divisor = 0;

try {
    // Pode gerar ArithmeticException ou IndexOutOfBounds
    int calculo = numeros[5] / divisor;
}
catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
    // Tratamento unificado para ambas as falhas
    System.out.println("Erro de limite de array ou matemática!");
}
```


NotebookLM

A Cláusula Incondicional: O Bloco Finally

O bloco **finally** cria uma garantia de execução. Ele será executado sempre, independentemente de o bloco try ter terminado normalmente ou de uma exceção ter sido capturada.



Uso Estratégico: Essencial para liberar recursos do sistema, como fechar conexões de banco de dados, fechar scanners ou arquivos abertos.

NotebookLM

Exemplo Prático: Try-Catch-Finally

```
try {
    System.out.println("1. Abrindo conexão...");
    int erro = 10 / 0; // Gera exceção
} catch (Exception e) {
    System.out.println("2. Tratando a falha.");
} finally {
    System.out.println("3. Fechando conexão garantidamente.");
}
```

```
> 1. Abrindo conexão...
> 2. Tratando a falha.
> 3. Fechando conexão garantidamente.
```


NotebookLM

Assumindo o Controle: A Instrução throw



Até agora, a JVM gerou as exceções. Com a instrução **throw**, o programador pode instanciar e lançar manualmente uma exceção quando uma regra de negócio for violada.

```
throw new IllegalArgumentException("Valor inválido!");
```

[Ação de lançar] [O tipo do objeto gerado] [A mensagem de diagnóstico]

NotebookLM

Exemplo Prático: throw

```
public void cadastrarUsuario(int idade) {
    if (idade < 18) {
        // Lança a exceção manualmente bloqueando o fluxo
        throw new IllegalArgumentException("Idade deve ser >= 18");
    }
    System.out.println("Usuário cadastrado com sucesso!");
}
```

Ao encontrar o throw, o método é imediatamente interrompido. Nenhuma linha de código abaixo dele será executada.

NotebookLM

Delegando Responsabilidades: A Cláusula throws

Quando um método sabe que pode gerar um erro, mas não tem a intenção (ou contexto) para tratá-lo, ele usa throws em sua assinatura.



O Aviso Prévio: O throws funciona como um aviso formal: "Este método é perigoso. Se você quiser chamá-lo, a responsabilidade de usar o try-catch é toda sua."

NotebookLM

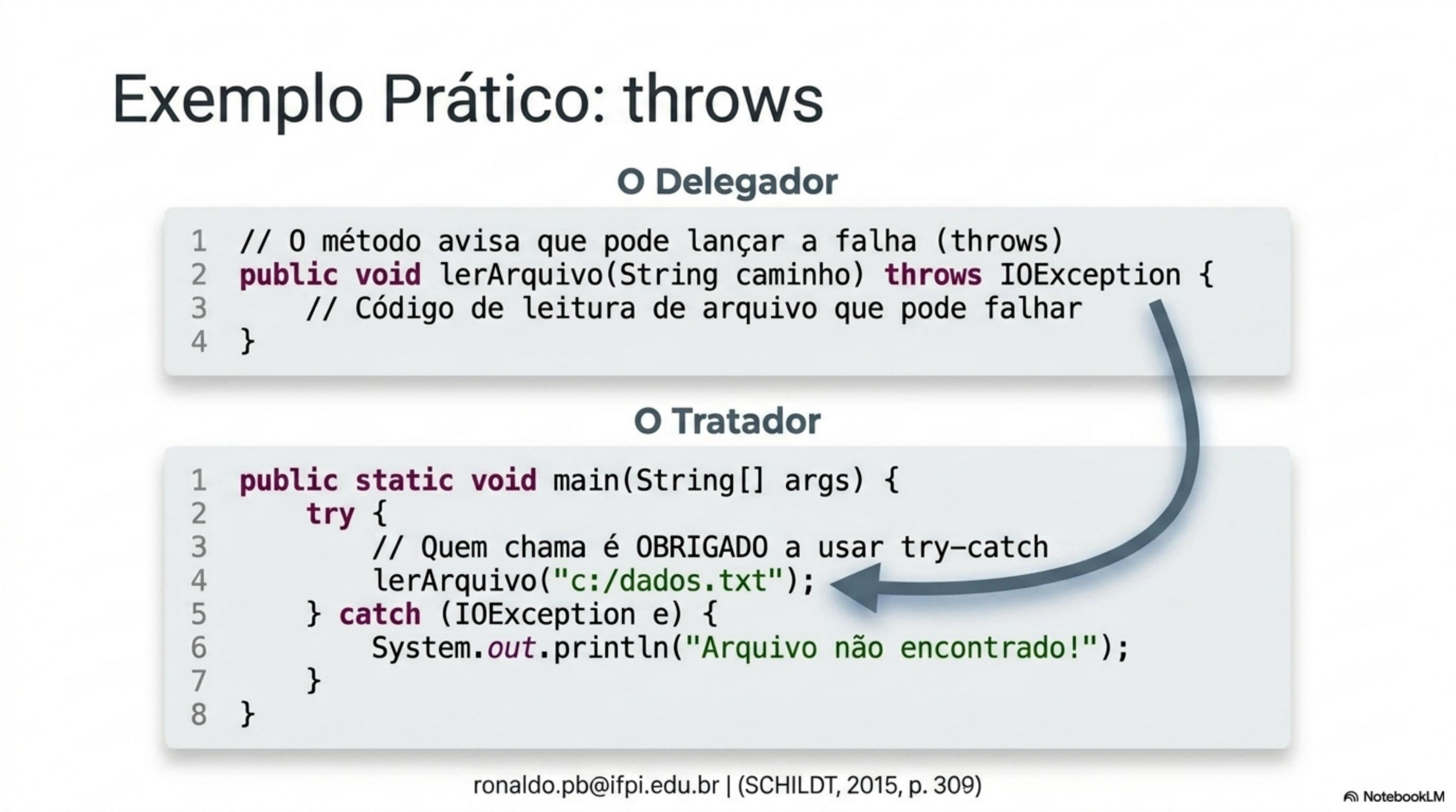
Exemplo Prático: throws

O Delegador

```java
// O método avisa que pode lançar a falha (throws)
public void lerArquivo(String caminho) throws IOException {
    // Código de leitura de arquivo que pode falhar
}
```

O Tratador

```java
public static void main(String[] args) {
    try {
        // Quem chama é OBRIGADO a usar try-catch
        lerArquivo("c:/dados.txt");
    } catch (IOException e) {
        System.out.println("Arquivo não encontrado!");
    }
}
```



NotebookLM

Referências Bibliográficas

DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

SCHILDT, Herbert. Java para iniciantes: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015.

TERUEL, Evandro Carlos. Programação Orientada a Objetos com Java – sem mistérios. São Paulo, 2016.

NotebookLM